《算法分析与设计》

课后作业

学　　号　2024112639

姓　　名　毛露涵

年　　级　2024 级

专　　业　软件工程

院　　系　计算机与人工智能学院

第一大题（40%）	第二大题（40%）	第三大题（20%）	第四大题（0%）	总分	批阅人签字

2025-2026 学年第 2 学期

第 5 章 动态规划算法

一、编程题 1（共 40 分）

某公司进行组合投资，把钱同时投入到多个项目，并希望获取最大的收益。在已知该公司希望投资的最大金额数和投资的项目数的情况下，请编写程序计算其所能获得的最大收益。

输入：

输入的第一行包含两个整数 n 和 m，分别表示该公司的投资资金数和希望投资的项目数。其后的 n 行，每行有 m 个整数，分别表示将金额 1、2、…、n 单独投资到 m 个项目时的收益（投资为 0 时 m 个项目的收益均为 0）。每次组合投资必须将 n 用完，组合投资中某个项目的投资额可以为 0。

输出：

输出一行，表示该公司进行组合投资所能获得的最大收益。

样例输入：

3 2

3 2

3 5

4 7

样例输出：

8

要求：

（1） 写出采用动态规划法求解该问题的递推方程和边界条件。（10 分）

设 val[k][i]表示把金额 K单纯投到第 i个项目的收益，val[0][i]=0

dp[i][j]表示把金额j分配给前i个项目时的最大收益

递推方程：$dp[i][j]=\max\{dp[i-1][j-k]+val[k][i]\}$，$1<=i<=m$，$0<=j<=n$，$0<=k<=j$

边界条件：$dp[0][0]=0$

$dp[0][j]=-\infty\ (j>0)$

$dp[i][0]=0$

（2） 用动态规划法分析样例输入的求解过程，手工计算 dp 数组的值。（10 分）

只投资前1个项目：dp[1][0]=0

dp[1][1]=3

dp[1][2]=3

dp[1][3]=4

投资前2个项目：dp[2][0]=0

dp[2][1]=max(dp[1][1],dp[1][0]+2)=3

dp[2][2]=max(dp[1][2],dp[1][1]+2,dp[1][0]+5)=5

dp[2][3]=max(dp[1][3],dp[1][2]+2,dp[1][1]+5,dp[1][0]+7)=8

最终dp数组

i\j	0	1	2	3
dp[0][j]	0	-∞	-∞	-∞
dp[1][j]	0	3	3	4
dp[2][j]	0	3	5	8

最大收益为8

（3） 采用动态规划法编写程序实现上述题目要求，附源代码。（15 分）

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    int n,m;
    cin>>n>>m;
    vector<vector<long long>>val(n+1,vector<long long>(m+1,0));
    for(int i=1;i<=n;i++){
        for(int j=1;j<=m;j++){
            cin>>val[i][j];
        }
    }
    const long long neg=-4e18;
    vector<vector<long long>>dp(m+1,vector<long,long>(n+1,neg));
    dp[0][0]=0;
    for(int=1;i<=m;i++){
        dp[i][0]=0;
        for(int j=1;j<=n;j++){
            for(int k=0;k<=j;k++){
                dp[i][j]=max(dp[i][j],dp[i-1][j-k]+val[k][i]);
            }
        }
    }
    cout<<dp[n][n]<<'\n';return 0;
}
```

源代码：

```
#include <bits/stdc++.h>
using namespace std;

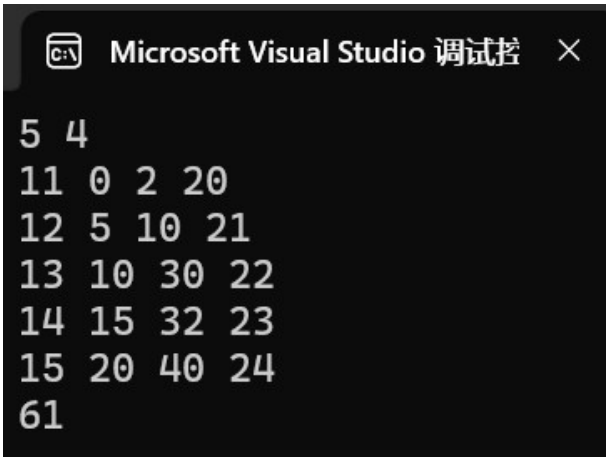
int main()
{
    int n, m;
```

```
    cin >> n >> m;
    vector<vector<long long>> val(n + 1, vector<long long>(m + 1, 0));
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            cin >> val[i][j];
        }
    }
    const long long neg = -4e18;
    vector<vector<long long>> dp(m + 1, vector<long long>(n + 1, neg));
    dp[0][0] = 0;
    for (int i = 1; i <= m; i++) {
        dp[i][0] = 0;
        for (int j = 1; j <= n; j++) {
            for (int k = 0; k <= j; k++) {
                dp[i][j] = max(dp[i][j], dp[i - 1][j - k] + val[k][i]);
            }
        }
    }
    cout << dp[m][n] << '\n';
    return 0;
}
```

（4）计算以下测试用例的输出，附屏幕截图。（5 分）

```
5 4
11 0 2 20
12 5 10 21
13 10 30 22
14 15 32 23
15 20 40 24
```

测试输出：



二、编程题 2（共 40 分）

有一天美羊羊正在草地上玩耍，突然天上开始落金币，这些金币掉落的范围在一个固定的水平区域内，但这些金币一旦掉落到地上就消失了，因此美羊羊只有不断地移动并从空中接住这些金币才能得到它们。假设金币掉落的位置为 0 开始到 10 这 11 个位置，美羊羊开始时站在第 5 个位置，它可以以每秒 1 个位置的速度左右移动到相邻的位置，并接住掉落的金币。请问美羊羊最多能接住多少个金币？假设它一旦接住这些金币就不会掉落到地上。



输入：

输入数据有多组。每组数据的第一行为一个正整数 n（0<n<100000），表示有 n 个金币掉落在这个区域。在接下来的 n 行中，每行有两个整数 x，t（0<t<100000），表示在第 t 秒有一个金币掉落在 x 的位置上。同一秒钟在同一位置上可能掉下多个金币，n=0 时输入结束。

输出：

每一组输入数据对应一行输出。输出一个整数 m，表示美羊羊最多可能接到 m 个金币。

样例输入：

6

5 1

4 1

6 1

7 2

7 2

8 3

0

样例输出：

4

要求：

（1） 写出采用动态规划法求解该问题的递推方程和边界条件。（10 分）

设cnt[t][x]表示第t秒位置x掉落的金币数

dp[t][x]表示第t秒美羊羊站在位置x时，最多能接到的金币数

递推方程：$dp[t][x]=cnt[t][x]+\max(dp[t-1][x-1],dp[t-1][x],dp[t-1][x+1]),0<=x<=10,1<=t<=maxT$

边界条件：$dp[0][5]=0$

$dp[0][x]=-\infty, x!=5$

（2） 用动态规划法分析样例输入的求解过程，手工计算 dp 数组的值。（10 分）

初始状态：

t\x	0	1	2	3	4	5	6	7	8	9	10
0	-∞	-∞	-∞	-∞	-∞	0	-∞	-∞	-∞	-∞	-∞

第1秒

t\x	0	1	2	3	4	5	6	7	8	9	10
1	-∞	-∞	-∞	-∞	1	1	1	-∞	-∞	-∞	-∞

第2秒

t\x	0	1	2	3	4	5	6	7	8	9	10
2	-∞	-∞	-∞	1	1	1	1	3	-∞	-∞	-∞

第3秒

t\x	0	1	2	3	4	5	6	7	8	9	10
3	-∞	-∞	1	1	1	1	3	3	4	-∞	-∞

$\therefore \max(dp[3][x])=4$

最多接到4个

（3） 采用动态规划法编写程序实现上述题目要求，附源代码。（15 分）

```
#include<bits/stdc++.h>
using namespace std;
int main(){
  int n;
  while(cin>>n&&n!=0){
    vector<vector<int>>cnt(10000,vector<int>(11,0));
    int maxT=0;
    for(int j=0;j<n;j++){
      int x,t;
      cin>>x>>t;
      cnt[t][x]++;
      maxT=max(maxT,t);
    }
    const int neg=-1000000000;
    vector<vector<int>>dp(maxT+1,vector<int>(11,neg));
    dp[0][5]=0;
    for(int t=1;t<=maxT;t++){
      for(int x=0;x<=10;x++){
        int best=dp[t-1][x];
        if(x>0){
          best=max(best,dp[t-1][x-1]);
        }
        if(x<10){
```

```
            best=max(best,dp[t-1][x+1]);
        }
        dp[t][x]=best+cnt[t][x];
    }
}
int ans=0;
for(int x=0; x<=10; x++){
    ans=max(ans,dp[maxT][x];
}
cout<<ans<<'\n';
}
return 0;
}
```

源代码：

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    while (cin >> n && n != 0) {
        vector<vector<int>> cnt(100001, vector<int>(11, 0));
        int maxT = 0;
        for (int i = 0; i < n; i++) {
            int x, t;
            cin >> x >> t;
            cnt[t][x]++;
            maxT = max(maxT, t);
        }
        const int neg = -1000000000;
        vector<vector<int>> dp(maxT + 1, vector<int>(11, neg));
        dp[0][5] = 0;
        for (int t = 1; t <= maxT; t++) {
            for (int x = 0; x <= 10; x++) {
                int best = dp[t - 1][x];
```

```
                    if (x > 0) {
                        best = max(best, dp[t - 1][x - 1]);
                    }
                    if (x < 10) {
                        best = max(best, dp[t - 1][x + 1]);
                    }
                    dp[t][x] = best + cnt[t][x];
                }
            }
            int ans = 0;
            for (int x = 0; x <= 10; x++) {
                ans = max(ans, dp[maxT][x]);
            }
            cout << ans << '\n';
        }
        return 0;
}
```

（4）给出由程序算出的上述测试用例的输出，附屏幕截图（5 分）。

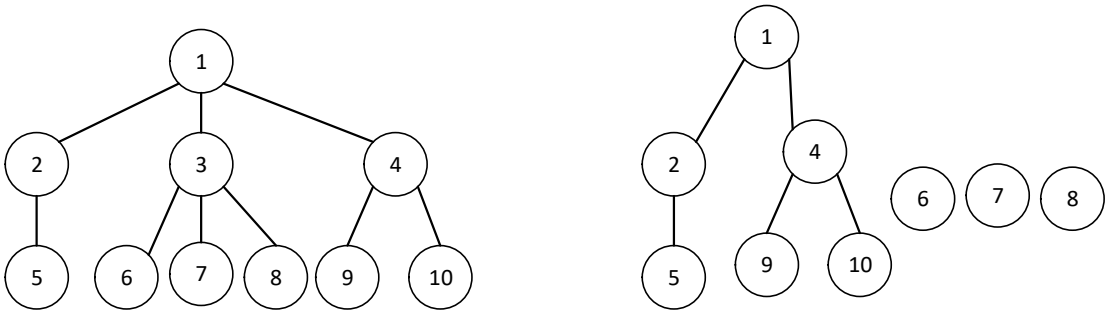
测试输出：



三、编程题 3（共 20 分）

定义树中某结点的平衡为删除该结点后所形成的森林中各树结点数的最大

值。如下图左边的树，如果删除其中的一个结点 3，则该树变成右边的森林，也就是多棵树，这些树的结点数分别为 6，1，1，1，其最大值为 6，因此结点 3 的平衡为 6。



在给定一棵树的情况下，求该树中所有结点的最小平衡数。

输入：

输入的第一行是一个整数 n（(1 <= n<= 20)，表示测试数据的组数。每一组测试数据的第一行是一个整数 m (1 <= m <= 20,000)，表示树中边的数量，其后的 m 行，每一行包含两个整数，表示树中相连的两个结点。树中结点的编号从 1 开始编号。

输出：

对于每一组测试数据输出一行，表示该树中所有结点的平衡数的最小值。

输入样例：

1
9
1 2
1 3
1 4
2 5
3 6
3 7
3 8
4 9
4 10

输出样例：

4

要求：

（1） 分析样例输入的求解过程，手工得到计算结果。（10 分）

删除结点 1：
形成 3 棵树：
{2,5}，结点数为 2
{3,6,7,8}，结点数为 4
{4,9,10}，结点数为 3
所以结点 1 的平衡数为 max(2,4,3)=4
删除结点 2：
形成 2 棵树：
{5}，结点数为 1
{1,3,4,6,7,8,9,10}，结点数为 8
所以结点 2 的平衡数为 8
删除结点 3：
形成 4 棵树：
{6}，结点数为 1
{7}，结点数为 1
{8}，结点数为 1
{1,2,4,5,9,10}，结点数为 6
所以结点 3 的平衡数为 6
删除结点 4：
形成 3 棵树：
{9}，结点数为 1
{10}，结点数为 1
{1,2,3,5,6,7,8}，结点数为 7
所以结点 4 的平衡数为 7
删除结点 5、6、7、8、9、10：
它们都是叶子结点，删除后剩下的树都有 9 个结点
所以这些结点的平衡数都是 9
因此所有结点的平衡数为：
结点 1：4
结点 2：8
结点 3：6
结点 4：7
结点 5：9
结点 6：9

结点 7：9
结点 8：9
结点 9：9
结点 10：9
最小平衡数为：4

（2） 编写程序实现上述题目要求，并给出上述测试用例的输出。（10 分）

源代码：

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int t;
    cin >> t;

    while (t--) {
        int m;
        cin >> m;

        int nodeCnt = m + 1;
        vector<vector<int>> g(nodeCnt + 1);

        for (int i = 0; i < m; i++) {
            int x, y;
            cin >> x >> y;
            g[x].push_back(y);
            g[y].push_back(x);
        }

        vector<int> parent(nodeCnt + 1, 0);
        vector<int> sub(nodeCnt + 1, 1);
        vector<int> order;
```

```
stack<int> st;
st.push(1);
parent[1] = -1;

while (!st.empty()) {
    int u = st.top();
    st.pop();
    order.push_back(u);

    for (int v : g[u]) {
        if (v == parent[u]) continue;
        parent[v] = u;
        st.push(v);
    }
}

for (int i = nodeCnt - 1; i >= 0; i--) {
    int u = order[i];
    for (int v : g[u]) {
        if (parent[v] == u) {
            sub[u] += sub[v];
        }
    }
}

int ans = nodeCnt;

for (int u = 1; u <= nodeCnt; u++) {
    int bal = nodeCnt - sub[u];

    for (int v : g[u]) {
        if (parent[v] == u) {
            bal = max(bal, sub[v]);
```

```
                }
            }

            ans = min(ans, bal);
        }

        cout << ans << '\n';
    }

    return 0;
}
```

执行结果：

